



[Список тренингов](#) / [Промпт-инжиниринг](#) / [Промпт-инжиниринг 2.0](#) / [Модуль 10. Кейсы](#)
[\(/teach/control/stream/view/id/931478155\)](/teach/control/stream/view/id/931478155)

PErr10.7. ИИ-ассистент. Прескоринг кандидатов на вакансии (new)

Предыдущий урок

[\(/pl/teach/control/lesson/view?id=341528207\)](/pl/teach/control/lesson/view?id=341528207)

Следующий урок

[\(/pl/teach/control/lesson/view?id=341528463\)](/pl/teach/control/lesson/view?id=341528463)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Уважаемые студенты!

В рамках данного урока могут рассматриваться примеры передачи персональных данных при регистрации на иностранных сервисах, а также обработки и хранения пользовательских данных.

В соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных», обращаем ваше внимание: в демонстрационных примерах и при выполнении домашних

заданий необходимо использовать исключительно тестовые данные или тестовые аккаунты, в которых не указаны персональные данные — используйте вымышленные ники, дополнительные данные и т.п.

Использование иностранных сервисов не является обязательным. Вы вправе выполнять домашние задания на аналогичных разрешённых платформах по вашему выбору.

Что мы умеем

- ✓ Проектируем систему автоматического прескоринга кандидатов на базе GPT.
- ✓ Разрабатываем структурированные промпты для анализа резюме и сравнения с вакансией.
- ✓ Организуем получение строго валидного JSON-ответа.
- ✓ Реализуем парсинг PDF-резюме с помощью pdfplumber и подготавливаем текст к анализу.
- ✓ Настраиваем подключение к API через .env и интеграцию с OpenAI.
- ✓ Создаем веб-интерфейс на Streamlit для загрузки вакансии и резюме.
- ✓ Реализуем расчет итогового score по модели весов (hard skills, experience, soft skills).
- ✓ Сохраняем историю кандидатов и обеспечиваем их сравнение внутри системы.
- ✓ Строим полноценный HR-инструмент как production-прототип для портфолио.

Результат дня

Ты создашь полноценный веб-сервис автоматического прескоринга кандидатов:

- загрузка вакансии и PDF-резюме;
- интеллектуальный анализ через GPT;
- расчет итогового score по 100-балльной шкале;
- выделение сильных и слабых сторон кандидата;
- хранение и сравнение результатов в интерфейсе.

В результате у тебя будет рабочий HR-инструмент, который можно адаптировать под реальные бизнес-процессы или добавить в портфолио как практический кейс автоматизации.

▶ Время чтения и просмотра: ~ 2 часа

🕒 Время выполнения задания: ~ 2 часа

Теория на сегодня

1. ИИ-прескоринг кандидатов: архитектура, промпты и реализация веб-сервиса

ИИ-Ассистент:
преск
кандид

Получить персональное
видео

zerocoder.ru

Продолжаем изучение курса **Prompt Engineering**. Сегодня создадим ИИ-ассистента для прескоринга кандидатов — систему, которая поможет HR-специалистам проводить предварительную оценку перед интервью.

Сегодня мы:

1. Научимся использовать GPT для анализа резюме.
2. Разработаем промпты для сравнения вакансии и кандидата.
3. Создадим интерфейс приложения на Streamlit.
4. Реализуем автоматический прескоринг кандидатов.

▼ Введение

✓ **Цель текущего урока** — использовать **GPT** для анализа резюме и создания системы автоматического прескоринга кандидатов.

Будет реализовано:

- чтение данных из **PDF-файла** (резюме кандидата);
- анализ содержимого с помощью **GPT**;
- сравнение с текстом вакансии;
- формирование итогового score;
- выделение сильных и слабых сторон кандидата.

✓ **Прескоринг** — это предварительная оценка кандидата на соответствие вакансии.

У компании есть:

- описание вакансии;
- список требований (опыт, навыки, технологии, сфера проектов);
- ожидания по образованию и уровню.

Перед приглашением на интервью необходимо понять:

- относится ли кандидат к нужной сфере;
- есть ли у него релевантный опыт;
- владеет ли он необходимыми технологиями;

- соответствует ли его профиль уровню позиции.

⚠ Если кандидат не соответствует базовым требованиям, приглашение на собеседование нецелесообразно.

Техническая реализация (в проекте будет):

1. Разработка промптов для:

- анализа резюме;
- структурирования информации;
- сравнения кандидата с вакансией;
- расчёта итогового score.

2. Создание интерфейса на **Streamlit**.

✓ **Streamlit** — это библиотека **Python** для быстрого создания веб-интерфейсов.

Проведем сравнение:

- в чём **Streamlit** проще, чем **Flask**;
- где он быстрее в разработке;
- какие ограничения существуют по сравнению с **Flask**.

3. В результате будет создан веб-сервис прескоринга, который позволяет:

- загрузить текст вакансии;
- загрузить PDF-резюме кандидата;
- автоматически получить оценку соответствия;
- увидеть сильные и слабые стороны кандидата.

Это полноценный кейс автоматизации **HR-процесса**, который можно добавить в портфолио как практический проект.

▼ Прескоринг ↘

✓ **Прескоринг** — это предварительная автоматизированная оценка кандидата на соответствие вакансии.

Термин можно разобрать на две части:

- **Pre** — до, предварительно;
- **Scoring** — оценка, присвоение баллов.

Следовательно, прескоринг — это система предварительной оценки.

Систему можно условно разделить на два блока:

1. Сравнение требований вакансии и резюме кандидата, при котором анализируется:

- релевантный опыт;
- технологический стек;
- сферы проектов;
- образование;
- глубина погружения в технологии.

2. Формирование итогового score

Кандидату присваивается числовая оценка, отражающая степень соответствия конкретной вакансии.

⚠️ Высокий **score** не означает, что кандидат «сильнее» другого в абсолютном смысле. Это означает лишь то, что его опыт и навыки более релевантны именно данной вакансии.

Система позволяет:

- прогонять несколько кандидатов через единый критерий оценки;
- сравнивать их между собой в рамках одной позиции;
- определить кандидата №1 для приглашения на собеседование.

⚠️ **Прескоринг** экономит время и помогает принимать решения на основе структурированного анализа, а не субъективного впечатления.

▼ Ключевые аспекты ручного отбора кандидатов

Сегодня решается проблема ручного отбора кандидатов. У этой проблемы есть три ключевых аспекта.

1. Большое количество резюме

На одну вакансию может приходиться сотни, а иногда и тысячи откликов.



Ручная проверка такого объёма — это значительная нагрузка.

При этом значительная часть кандидатов (до 80–90%) не соответствует базовым требованиям:

- отсутствует необходимый опыт (например, позиция middle требует подтверждённого опыта работы);
- не совпадает направление (например, frontend разработчик откликается на вакансию backend);
- отсутствует нужный технологический стек.

Эти кандидаты могут быть автоматически отсеяны на этапе предварительной оценки, без ручного анализа. Это существенно разгружает HR-специалиста.

2. Разные форматы резюме

Даже если большинство резюме представлено в **PDF**, структура у всех разная:

- разные блоки;
- разный порядок информации;
- различный дизайн;
- наличие или отсутствие фотографий, таблиц и т.д.

Человеку сложно часами подряд читать тексты в разных форматах. Возникает усталость, внимание рассеивается, появляется риск пропустить важную информацию.



Автоматизированная система не учитывает визуальное оформление. Для неё важен только текст и его содержимое. Анализ проводится по смыслу, а не по структуре документа.

3. Человеческий фактор

При длительной работе с большим количеством однотипных документов неизбежны ошибки:

- кандидат может быть ошибочно отсеян;
- кандидат может пройти дальше, хотя не соответствует требованиям.

Человеческий фактор полностью исключить невозможно. Однако можно уменьшить количество ситуаций, в которых он проявляется.

⚠ Если специалист проверяет 1000 резюме и допускает ошибку в 5% случаев — это 50 ошибок.

Если через автоматический прескоринг проходит 100 резюме — потенциальных ошибок становится 5.

Процент ошибки остаётся тем же, но абсолютное количество снижается.

Автоматизированный прескоринг:

- сокращает объём ручной работы;
- снижает нагрузку на **HR**;
- уменьшает влияние человеческого фактора;
- ускоряет процесс подбора.

В результате весь алгоритм найма становится быстрее и структурированнее.

▼ Архитектура решения

Рассмотрим общую логику системы, по сути **pipeline** обработки данных.

1. Ввод вакансии

Для оценки кандидата необходим текст вакансии. Система должна понимать:

- на какую позицию подаётся кандидат;
- какие требования предъявляются;
- какие навыки и опыт являются обязательными.

⚠ Поэтому первым шагом будет ввод текста вакансии.

2. Загрузка резюме

Формат — **PDF**, потому что:

- подавляющее большинство резюме приходит в этом формате;
- это стандарт де-факто в HR-процессах.



Если система умеет корректно работать с **PDF**, она покрывает основной объём реальных кейсов.

В дальнейшем решение можно расширить:

- подключить получение резюме через **API** (например, **hh**);
- добавить поддержку других форматов;
- подключить **OCR** при работе с изображениями;
- адаптировать систему под конкретный бизнес-процесс компании.



Формат входящих данных всегда зависит от конкретной задачи бизнеса.

3. Анализ через GPT

На этом этапе модель:

- читает текст вакансии;
- читает текст резюме;
- сопоставляет требования и опыт;
- выделяет сильные стороны кандидата;
- выделяет слабые стороны;
- рассчитывает итоговый **score**.

4. Расчёт score

Score будет рассчитываться по 100-балльной шкале: чем ближе значение к 100, тем выше релевантность кандидата данной вакансии.



Это оценка соответствия конкретной позиции, а не абсолютная оценка уровня специалиста.

5. Вывод результата

Система возвращает:

- итоговый балл;

- список сильных сторон;
- список слабых сторон;
- краткое резюме соответствия.

Результаты можно сохранить или использовать дальше в автоматизированном **HR-процессе**.

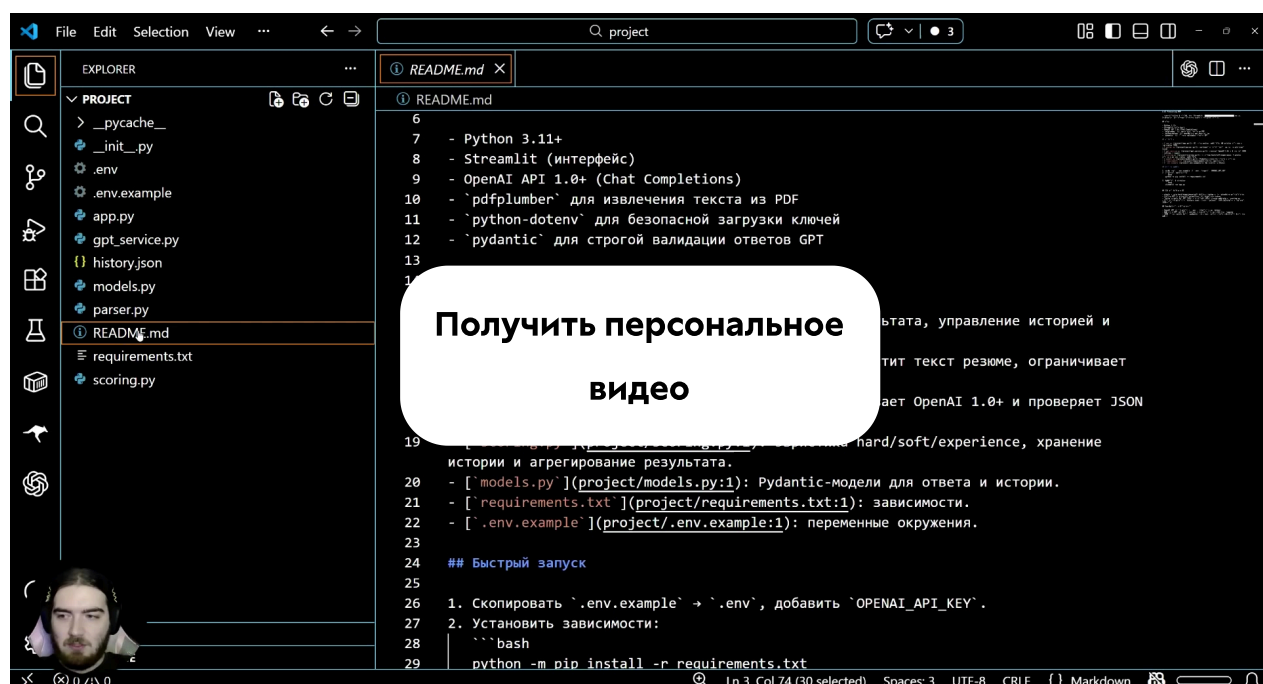
Вся логика проста:

1. Ввод вакансии
2. Загрузка резюме
3. Анализ
4. Вывод результата

При необходимости систему можно:

- доработать;
- изменить логику расчёта **score**;
- добавить дополнительные параметры оценки;
- адаптировать под собственные проекты.

2. Внутреннее устройство проекта: код, API, модели и система оценки



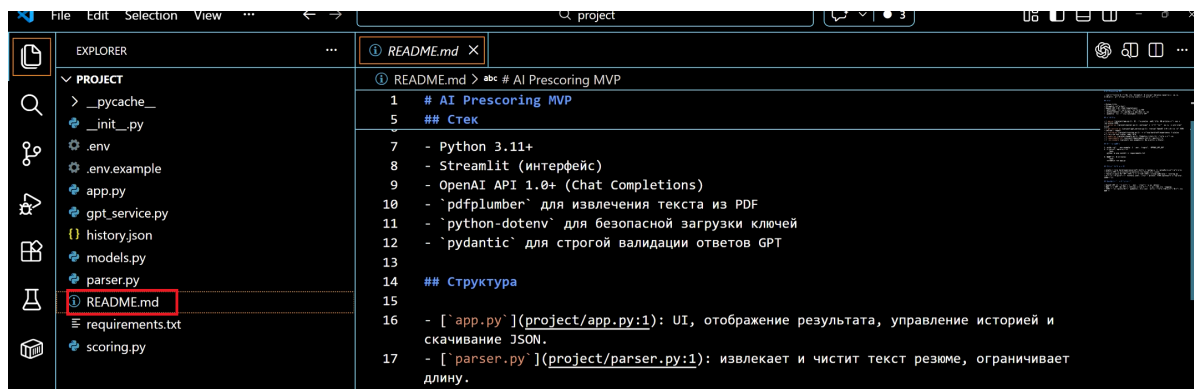
Рассмотрим структуру проекта и ключевые файлы, необходимые для запуска и работы системы прескоринга.

1. Доступ к проекту

- Проект размещен на **GitHub**.
- По ссылке можно изучить структуру, скачать и запустить локально.

2. README-файл

- Содержит краткое описание проекта.
- В дальнейшем описание может быть расширено.
- В **README** указана вся необходимая информация: структура проекта; порядок запуска; описание файлов.
- При возникновении вопросов по запуску или устройству проекта необходимо обращаться к **README**.



3. Описание проекта

Проект — производственный прототип. Интерфейс реализован на **Streamlit**. Принимает описание вакансии и резюме. Композитно рассчитывает итоговый скор. Сохраняет историю результатов.

Хранение истории важно, так как работа ведется с множеством кандидатов, и требуется параллельно видеть их оценки.

4. Технологический стек:

- **Python 3.11**
- **Streamlit** — интерфейс
- **OpenAI API** — запросы к нейросети

⚠ **OpenAI** можно заменить на любого доступного провайдера (**GigaChat, YandexGPT, Perplexity** и др.). Для этого потребуется изменить подключение в коде и заменить

интеграцию с **OpenAI** на нужный **API**. Остальная логика проекта остается без изменений.

```
5  ## Стек
6
7  - Python 3.11+
8  - Streamlit (интерфейс)
9  - OpenAI API 1.0+ (Chat Completions)
10 - `pdfplumber` для извлечения текста из PDF
11 - `python-dotenv` для безопасной загрузки ключей
12 - `pydantic` для строгой валидации ответов GPT
13
```

5. Структура проекта:

app.py

- Точка входа.
- Запускает пользовательский интерфейс.
- Отображает результаты.
- Управляет историей.
- Позволяет скачивать **JSON**.
- Через этот файл происходит основное взаимодействие.

parser.py

- Извлекает и очищает текст из резюме.
- Ограничивает длину текста.
- Получает исключительно содержимое.
- Игнорирует форматирование, таблицы, визуальное оформление **PDF**.
- Работает только с текстом.

gpt_service.py

- Выполняет запрос к **OpenAI**.
- Получает ответ в формате **JSON**.
- Разбирает ответ.
- Передает данные для анализа и сохранения сора.
- Используется для оценки hard skills, soft skills и опыта.

models.py

- Используются для структуры ответов и хранения истории.

requirements

- Список зависимостей проекта.

.env.example

- Пример переменных окружения.
- Показывает, какие параметры должны быть заданы.

```
14  ## Структура
15
16  - [ `app.py` ](project/app.py:1): UI, отображение результата, управление историей и
    скачивание JSON.
17  - [ `parser.py` ](project/parser.py:1): извлекает и чистит текст резюме, ограничивает
    длину.
18  - [ `gpt_service.py` ](project/gpt_service.py:1): вызывает OpenAI 1.0+ и проверяет JSON
    с retries / repair.
19  - [ `scoring.py` ](project/scoring.py:1): эвристика hard/soft/experience, хранение
    истории и агрегирование результата.
20  - [ `models.py` ](project/models.py:1): Pydantic-модели для ответа и истории.
21  - [ `requirements.txt` ](project/requirements.txt:1): зависимости.
22  - [ `.env.example` ](project/.env.example:1): переменные окружения.
23
```

▼ Порядок настройки проекта

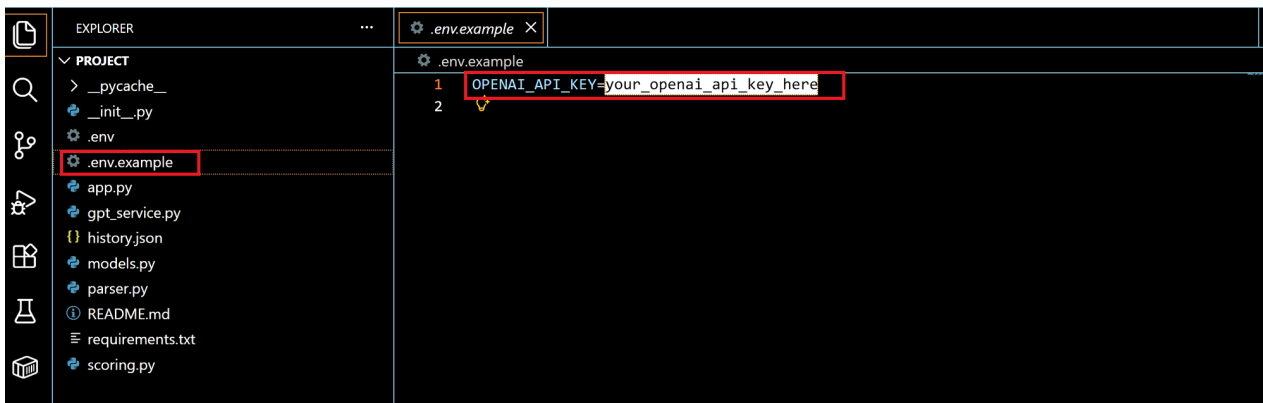
В проекте есть пример файла `.env.example` на **GitHub**. Файл используется для хранения ключей и настроек подключения к API. Необходимо заменить поля:

- Для **OpenAI** — `OpenAI_TOKEN`;
- Для других провайдеров — соответствующие уникальные ключи **API** (например, **GigaChat**: `ID`, `scope`, `user key`).

 Каждая нейросеть использует свои параметры, но принцип один:

- создать точку `.env`;
- заполнить её актуальными данными для подключения.

Файл `.env` является ключевым для запуска проекта. После корректного заполнения файл используется при старте и управляет подключением к выбранной нейросети. Это подготовительный этап перед запуском основного процесса проекта.

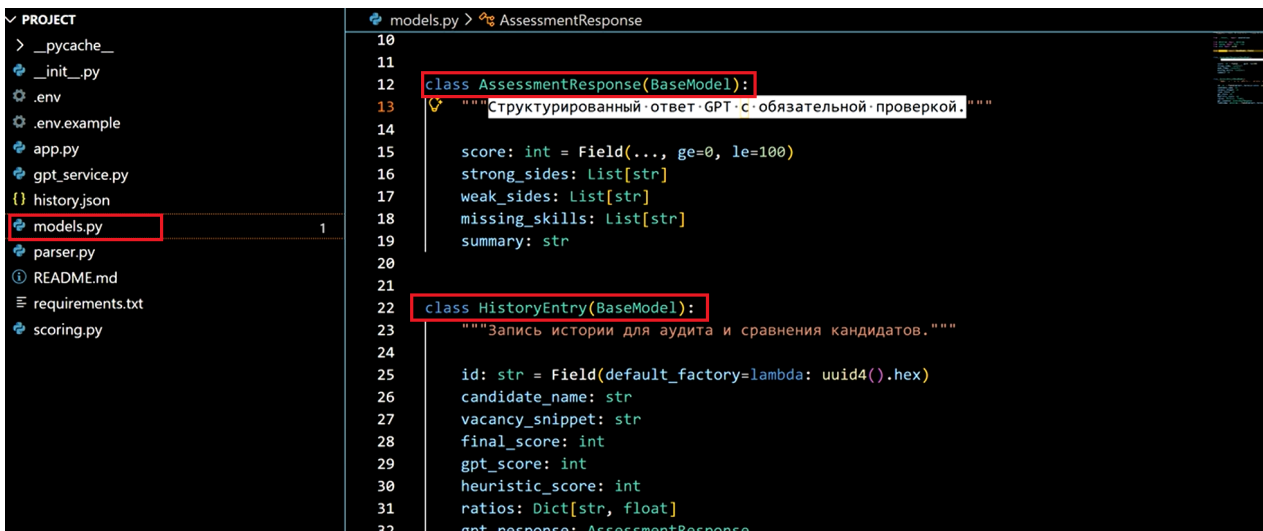


▼ Модели в проекте

Разберём использование моделей в проекте и их назначение для обработки запросов к нейросети и хранения результатов.

В проекте используются две модели:

- Структурированный ответ от **GPT**.
- Модель для записи истории и сравнения кандидатов.



1. Структурированный ответ от GPT

- При отправке запроса в нейросеть важно получать **структурированный ответ**, в формате **JSON**.
- Структура нужна, чтобы: результаты были организованы одинаково для всех кандидатов и можно было легко выделить ключевые элементы: общий счет, сильные стороны, слабые стороны, пропущенные моменты, общий вывод.
- Структура позволяет разбивать длинный текстовый ответ на удобные для чтения и анализа части.
- Разделенные элементы используются для дальнейшей визуализации и обработки на сайте.
- Промпт передается с настройками того, как модель должна отвечать.

2. Модель для записи истории

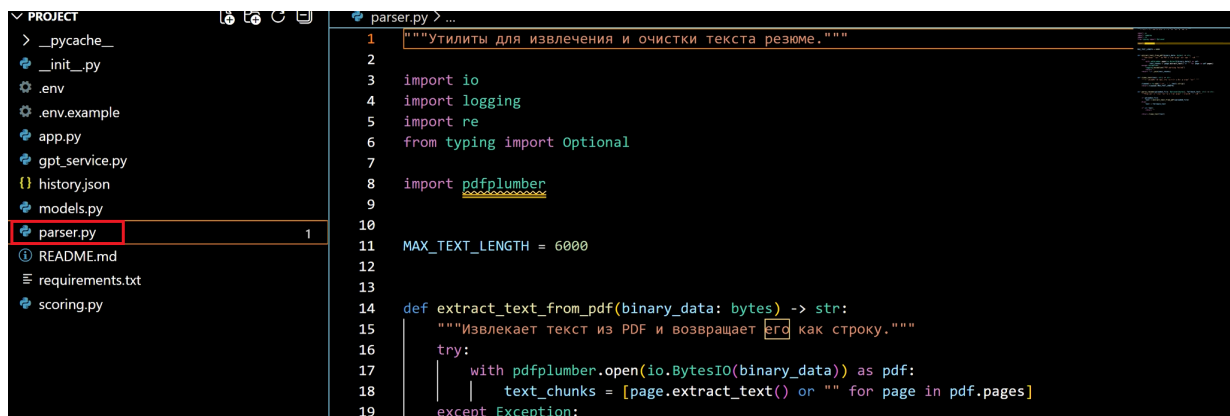
- Записывает результаты проверенных кандидатов.
- Позволяет сравнивать кандидатов между собой.
- Обеспечивает возможность вернуться к ранее проверенным кандидатам и проанализировать их заново.
- История помогает отслеживать лучший вариант и управлять данными в динамике.

▼ Процесс парсинга резюме

Рассмотрим процесс парсинга резюме из **PDF** и подготовку текста для обработки нейросетью.

1. Парсер резюме

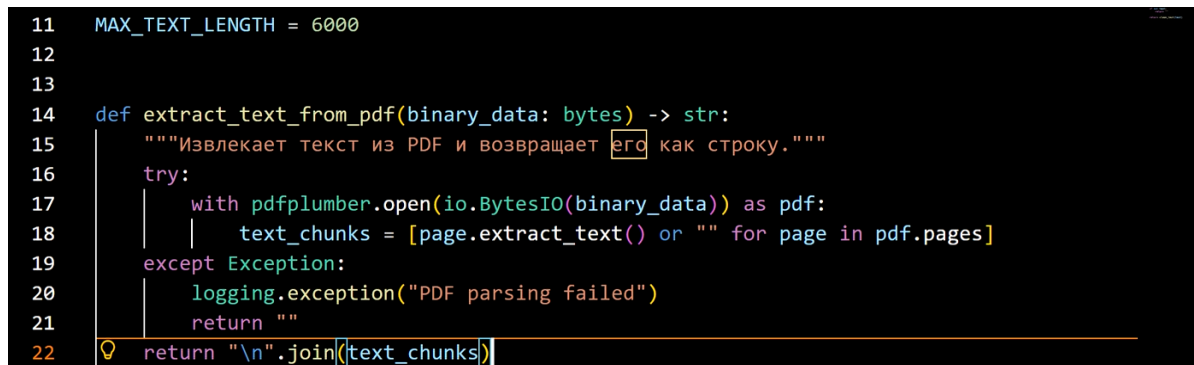
- Используется библиотека **pdfplumber** для извлечения текста из **PDF**.
- Ограничение длины текста — **6000 символов**. Это связано с ограничением контекста нейросети и этого объема достаточно для анализа большинства резюме.



```
1 """Утилиты для извлечения и очистки текста резюме."""
2
3 import io
4 import logging
5 import re
6 from typing import Optional
7
8 import pdfplumber
9
10
11 MAX_TEXT_LENGTH = 6000
12
13
14 def extract_text_from_pdf(binary_data: bytes) -> str:
15     """Извлекает текст из PDF и возвращает str как строку."""
16     try:
17         with pdfplumber.open(io.BytesIO(binary_data)) as pdf:
18             text_chunks = [page.extract_text() or "" for page in pdf.pages]
19     except Exception:
```

2. Этапы обработки текста

- **Извлечение текста.** Файл открывается и весь текст считывается в одну непрерывную строку. Убираются таблицы, переносы строк, визуальное оформление, лишние отступы и пробелы. Важно оставить только содержимое для анализа.



```
11 MAX_TEXT_LENGTH = 6000
12
13
14 def extract_text_from_pdf(binary_data: bytes) -> str:
15     """Извлекает текст из PDF и возвращает str как строку."""
16     try:
17         with pdfplumber.open(io.BytesIO(binary_data)) as pdf:
18             text_chunks = [page.extract_text() or "" for page in pdf.pages]
19     except Exception:
20         logging.exception("PDF parsing failed")
21         return ""
22     return "\n".join(text_chunks)
```

- **Очистка текста (клин текст)** Нормализуются невидимые символы: переносы строки, лишние пробелы, табуляции. Удаляется информационный шум, ненужный для анализа. Текст обрезается до максимум **6000** символов.

```
25 def clean_text(text: str) -> str:
26     """Нормализует пробельные символы и укорачивает текст."""
27
28     cleaned = re.sub(r"\s+", " ", text).strip()
29     return cleaned[:MAX_TEXT_LENGTH]
30
```

- **Объединение источников.** Текст можно получать либо из **PDF**, либо из дополнительного текстового поля (**full text**). Если текст отсутствует, возвращается пустая строка. Если текст есть, он очищается и возвращается готовым для анализа нейросетью.

```
32 def parse_resume(uploaded_file: Optional[bytes], fallback_text: str) -> str:
33     """Выбирает источник текста и возвращает очищенную строку."""
34
35     if uploaded_file:
36         text = extract_text_from_pdf(uploaded_file)
37     else:
38         text = fallback_text
39
40     if not text:
41         return ""
42
43     return clean_text(text)
44
```

⚠ Парсер обеспечивает чистый и структурированный текст. Текст готов для передачи в нейросеть для оценки кандидата. Основной процесс запускается через функцию **parse_resume**, которая объединяет все шаги.

▼ Процесс работы с сервисом GPT

Рассмотрим процесс работы с сервисом **GPT** для анализа резюме и подготовки структурированного ответа в формате **JSON**.

```
gpt_service.py > ...
9  import openai
10 from dotenv import load_dotenv
11 from openai import OpenAI, OpenAIError
12 from pydantic import ValidationError
13
14 from models import AssessmentResponse
15
16
17 load_dotenv()
18
19 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
20 CLIENT: Optional[OpenAI] = None
21
22 if not OPENAI_API_KEY:
23     logging.warning("OPENAI_API_KEY is not set. GPT service will not function correctly")
24 else:
25     CLIENT = OpenAI(api_key=OPENAI_API_KEY)
```

1.. Инициализация

- Получение **API-ключа** (**OpenAI** или другой провайдер).
- Создание клиента для отправки запросов.
- Настройка системного промта.

```

28 SYSTEM_PROMPT = (
29     "Ты профессиональный HR-аналитик. Оцени соответствие кандидата вакансии."
30     " Верни валидный JSON строго в формате, указанном в документации."
31     " Строковые поля не должны содержать переносов строк; если они нужны, используй '\\
32 )

```

2. Формат ответа

GPT (или любая другая модель) возвращает результат как длинную строку. В такой форме сложно анализировать отдельные части: **score**, сильные и слабые стороны, пропущенные навыки. Средние показатели по кандидатам или сравнение нескольких резюме становится неудобным.

Для удобной работы требуется структурированный **JSON**, содержащий:

- **score** — итоговый скоринг кандидата;
- **strengths** — сильные стороны;
- **weaknesses** — слабые стороны;
- **missing_skills** — пропущенные навыки;
- **summary** — общий вывод.

Это позволяет анализировать отдельные компоненты резюме и агрегировать результаты по кандидатам.

```

35 def _extract_json_payload(raw_text: str) -> str:
36     start = raw_text.find("{")
37     end = raw_text.rfind("}")
38     if start != -1 and end != -1 and end > start:
39         return raw_text[start : end + 1]
40     return raw_text
41

```

3. Ремонт структуры JSON

- Строговые поля не должны содержать обычные переносы строк `\n`.
- Переносы заменяются на `\\n`, чтобы **Python** корректно воспринимал строки в **JSON**.
- Это позволяет безопасно десериализовать данные и работать с ними как с объектами.

```

43 def _strip_code_block(text: str) -> str:
44     trimmed = text.strip()
45     if trimmed.startswith("```") and trimmed.endswith("```"):
46         return trimmed.strip("`\\n ")
47     return trimmed
48

```

4. Обработка JSON

- Все нестандартные или двойные кавычки заменяются на одинарные, чтобы **JSON** был корректно обработан.
- При необходимости добавляем недостающие кавычки. Если модель вернула ответ в формате с ``` или другими кавычками, эти блоки удаляются.
- Убираются escape-символы типа \\, которые могли остаться в тексте.
- Проверяется и корректируется остаточный слэш \, чтобы **Python** корректно прочитал **JSON**.

```

50 def _sanitize_json(raw: str) -> str:
51     buffer = []
52     in_string = False
53     escaped = False
54     for char in raw:
55         if char == '"' and not escaped:
56             in_string = not in_string
57         if char == '\\' and not escaped:
58             escaped = True
59             buffer.append(char)
60             continue
61         if char == '\r' and in_string:
62             buffer.append('\\r')
63             escaped = False
64             continue
65         if char == '\n' and in_string:
66             buffer.append('\\n')
67             escaped = False
68             continue
69         buffer.append(char)
70         escaped = False
71     return ''.join(buffer)

```

5. Repair JSON

- При первичной обработке строки из ответа нейросети могут быть потеряны или искажены кавычки, скобки и другие структурные элементы **JSON**. Без исправления дальнейшая работа с данными будет невозможна.
- Добавление отсутствующих кавычек или других необходимых символов для восстановления валидного **JSON**.
- Очистка лишних символов, оставшихся после преобразования строки (например, двойные слэши или кодовые блоки).
- Получаем **JSON**, который по структуре корректен, но хранится как строка.

```

74 def _repair_json(payload: str) -> str:
75     if payload.count("\\" % 2 != 0:
76         payload += "\"\"
77     open_braces = payload.count("{") - payload.count("}")
78     if open_braces > 0:
79         payload += "}" * open_braces
80     return payload
81

```

6. Передача запроса

- Определяется роль **System** и передается системный промпт (**System Prompt**).
- Промпт объясняет модели, что она — профессиональный аналитик, который оценивает соответствие кандидата вакансии.
- Указывается, что ответ должен быть строго в формате **JSON**.
- Обязательные поля: **score** — итоговый балл кандидата (тип **int**), **strong_sides** — список сильных сторон, **weak_sides** — список слабых сторон, **missing_skills** — список пропущенных навыков или пробелов в компетенциях, **summary** — общий вывод по резюме.
- Цель — получить структурированный **JSON**, с которым можно работать программно: анализировать, сравнивать кандидатов, хранить историю оценок. Избежать получения длинного неструктурированного текста, который сложно парсить и агрегировать.

```

83 def get_assessment(vacancy: str, resume: str, retries: int = 3) -> AssessmentResponse:
84     """Call OpenAI and validate the response against `AssessmentResponse`."""
85
86     if not vacancy or not resume:
87         raise ValueError("Vacancy and resume text are required to run GPT assessment.")
88
89     if CLIENT is None:
90         raise RuntimeError("OpenAI API key is missing; please configure OPENAI_API_KEY")
91
92     messages = [
93         {"role": "system", "content": SYSTEM_PROMPT},
94         {
95             "role": "user",
96             "content": (
97                 f"Вакансия:\n{vacancy.strip()}\n\n"
98                 f"Резюме:\n{resume.strip()}\n\n"
99                 "Верни ответ только как JSON в формате: {{\"score\": int, \"strong_side"
100                 f\"weak_sides\": [...], \"missing_skills\": [...], \"summary\": string}
101             ),
102         },

```

7. Модель GPT

- Используется **gpt-3.5-turbo** (достаточно для оценки соответствия текста резюме вакансии).
- Модели не требуется придумывать или генерировать сложный текст — только структурированная оценка.

- Можно заменить на любую доступную модель, поддерживающую аналогичный **API**.

```

105     for attempt in range(1, retries + 1):
106         try:
107             response = CLIENT.chat.completions.create(
108                 model="gpt-3.5-turbo",
109                 messages=messages,
110                 temperature=0.2,
111                 max_tokens=400,
112             )
113         except OpenAIError as error:
114             logging.exception("OpenAI API request failed")
115             if attempt == retries:
116                 raise
117             time.sleep(1)
118             continue
119 
```

8. Обработка JSON

- Проводятся все нужные операции, чтобы привести ответ в формат **JSON**.
- **JSON-объект** с оценкой кандидата готов к дальнейшему анализу и сохранению.

```

120     content = response.choices[0].message.content
121     payload = _strip_code_block(content)
122     payload = _extract_json_payload(payload)
123     payload = payload.strip()
124     payload = _sanitize_json(payload)
125     payload = _repair_json(payload)
126
127     try:
128         (variable) assessment: Any strict=False)
129         assessment = AssessmentResponse.parse_obj(parsed)
130         return assessment
131     except (json.JSONDecodeError, ValidationError) as error:
132         logging.warning(
133             "GPT assessment response validation failed on attempt %s: %s", attempt,
134         )
135         if attempt == retries:
136             raise RuntimeError("Failed to produce a valid assessment JSON after ret
137         time.sleep(1)
138         continue
139
140     raise RuntimeError("Unable to get assessment from GPT service.")
141 
```

▼ Система оценки кандидатов

1. Навыки разделены на три типа: жесткие (**hard skills**), софт (**soft skills**), опыт (**experience**).

- **Жесткие навыки** — технические, например, языки программирования и инструменты.
- **Опыт** — релевантность прошлой работы, важен, но не всегда передает реальные навыки.
- **Софт-навыки** — умение работать в команде, трудолюбие и т.д.

⚠ Распределение весов: жесткие 0,6, опыт 0,25, софт 0,15. Сумма = 1.

```
app.py
gpt_service.py
history.json
models.py
parser.py
README.md
requirements.txt
scoring.py

27
28
29 def _normalize_weights(weights: Dict[str, float]) -> Dict[str, float]:
30     """Нормализует веса, чтобы их сумма равнялась 1 и возвращает значения по умолчанию
31     total = sum(weights.values())
32     if not total:
33         logging.debug("Сумма весов равна нулю, используются значения по умолчанию.")
34         return {"hard": 0.6, "experience": 0.25, "soft": 0.15}
35     # Делим каждый вес на общую сумму, чтобы получить долю от общего веса
36     return {key: value / total for key, value in weights.items()}
37
38
```

2. Для каждого кандидата создается уникальный токен для сопоставления навыков. Используется при хранении и сравнении результатов оценки.

Система позволяет автоматически оценивать кандидатов по заранее заданной структуре весов. Обеспечивает сопоставимость оценок между кандидатами и хранение истории.

```
39 def _token_set(text: str) -> Iterable[str]:
40     """Извлекает уникальные токены длиной от четырех символов из текста для сопоставлен
41     return set(
42         token.lower()
43         for token in re.findall(r"\b[\wа-яёА-ЯЁ]{4,}\b", text)
44         if len(token) >= 4
45     )
46
```

3. Функции оценки:

- **Hard skills** определяет процент совпадений ключевых технических навыков кандидата с требованиями вакансии. Результат масштабируется в диапазон 0–100 на основе верхнего предела.
- **Experience** оценивает релевантность опыта кандидата по упоминаниям соответствующих навыков. Результат также в диапазоне 0–100.
- **Soft skills** определяет процент совпадений мягких навыков кандидата с ключевыми компетенциями. Считается по упоминаниям и сопоставлению с требуемыми навыками.

Происходит автоматическая оценка совпадений для каждого типа навыков. Унификация результатов для последующего суммирования и сравнения кандидатов.

```

_init_.py
.env
.env.example
app.py
gpt_service.py
history.json
models.py
parser.py
README.md
requirements.txt
scoring.py
OUTLINE
48 def calculate_hard_skill_score(vacancy: str, resume: str) -> float:
49     """Оценивает процент совпадений ключевых навыков между вакансией и резюме."""
50     vacancy_tokens = _token_set(vacancy)
51     resume_tokens = _token_set(resume)
52     if not vacancy_tokens:
53         return 0.0
54     match_count = len(vacancy_tokens & resume_tokens)
55     return round((match_count / len(vacancy_tokens)) * 100, 2)
56
57
58 def calculate_experience_score(resume: str) -> float:
59     """Извлекает годы опыта и масштабирует результат до диапазона 0-100 на основе верхн
60     years = [float(match) for match in re.findall(r"(\d+(?:\.\d+)?)\s*(?:years?|лет|yrs|
61     extracted = max(years, default=0.0)
62     scaled = min(extracted, 15.0) / 15.0
63     return round(scaled * 100, 2)
64
65
66 def calculate_soft_skill_score(resume: str) -> float:
67     """Определяет процент упоминаний мягких навыков в резюме по сравнению со списком кл
68     normalized_text = resume.lower()
69     matches = sum(1 for skill in SOFT_SKILLS if skill in normalized_text)
70     if not SOFT_SKILLS:
71         return 0.0
72     return round((matches / len(SOFT_SKILLS)) * 100, 2)
73
74

```

4. Для каждого типа навыков (**hard** , **experience** , **soft**) берется процент совпадений.

- Каждый процент умножается на вес навыка.
- Суммируются результаты для получения общего итогового балла.
- Возвращаются отдельно: баллы по каждому типу навыков, общий результат по кандидату.

```

75 def calculate_heuristic_score(
76     vacancy: str, resume: str, weights: Dict[str, float]
77 ) -> Dict[str, float]:
78     normalized_weights = _normalize_weights(weights)
79     hard = calculate_hard_skill_score(vacancy, resume)
80     experience = calculate_experience_score(resume)
81     soft = calculate_soft_skill_score(resume)
82     weighted = (
83         hard * normalized_weights["hard"]
84         + experience * normalized_weights["experience"]
85         + soft * normalized_weights["soft"]
86     )
87     return {
88         "hard": hard,
89         "experience": experience,
90         "soft": soft,
91         "heuristic": round(weighted, 2),
92         "ratios": normalized_weights,
93     }
94

```

5. Итоговый балл формируется как комбинация: эвристической оценки, гипотетической оценки, фиксированного коэффициента.

- Берется среднее между эвристикой и гипотетической оценкой.
- Результат сохраняется в истории оценок. Если файл истории отсутствует, то создается новый.
- Включены функции для добавления новых записей и обновления существующих.

```

96 def calculate_final_score(heuristic_score: float, gpt_score: int) -> int:
97     """Сочетает эвристическую и GPT-оценки по фиксированным коэффициентам для получения
98     final = (heuristic_score * 0.45) + (gpt_score * 0.55)
99     return min(100, max(0, int(round(final))))
100
101
102 def load_history() -> List[HistoryEntry]:
103     """Возвращает ранее сохраненные оценки, если файл истории доступен."""
104     if not HISTORY_FILE.exists():
105         return []
106     try:
107         with HISTORY_FILE.open(encoding="utf-8") as handler:
108             data = json.load(handler)
109             return [HistoryEntry.parse_obj(entry) for entry in data]
110     except Exception:
111         logging.exception("Не удалось загрузить историю оценок")
112     return []
113

```

▼ Интерфейс проекта

Файл `app.py` — это запуск и интерфейс проекта:

- **Используемая библиотека:** **Streamlit** для фронтенда и визуализации.

```

1 """Streamlit interface for the prescoring MVP."""
2
3 import json
4 import logging
5 from datetime import datetime
6 from typing import Dict
7
8 import streamlit as st
9
10 from parser import parse_resume
11 from scoring import (
12     append_history,
13     build_history_entry,
14     calculate_heuristic_score,
15     calculate_final_score,
16     load_history,
17 )
18 from gpt_service import get_assessment
19
20 logging.basicConfig(level=logging.INFO)

```

- **Основные функции интерфейса:** ввод текста вакансии — `text_area`. Ввод имени кандидата — `text_input`. Загрузка резюме — `file_uploader`.
- **Обработка баллов:** **<50** — неподходящий кандидат, **50-75** — частично подходящий, **75** — отличный результат (отображается зелёным).

```

23 def _score_color(score: int) -> str:
24     if score < 50:
25         return "#d63447"
26     if score < 75:
27         return "#f7b731"
28     return "#40c057"
29

```

- **Вывод истории:** используется `DataFrame` для таблицы кандидатов. Если история пуста — создается новый заголовок и таблица. Добавление новых записей через `append`.

```

58 def _display_history(history_entries) -> None:
59     if not history_entries:
60         st.info("История оценок пока пуста.")
61     return
62

```

- **Сравнение кандидатов:** предусмотрено в интерфейсе.
- **Main-функция:** Точка входа проекта, полностью запускает сайт. Настройка страницы: **Pageconfig** — название, описание, сайдбар с настройками.

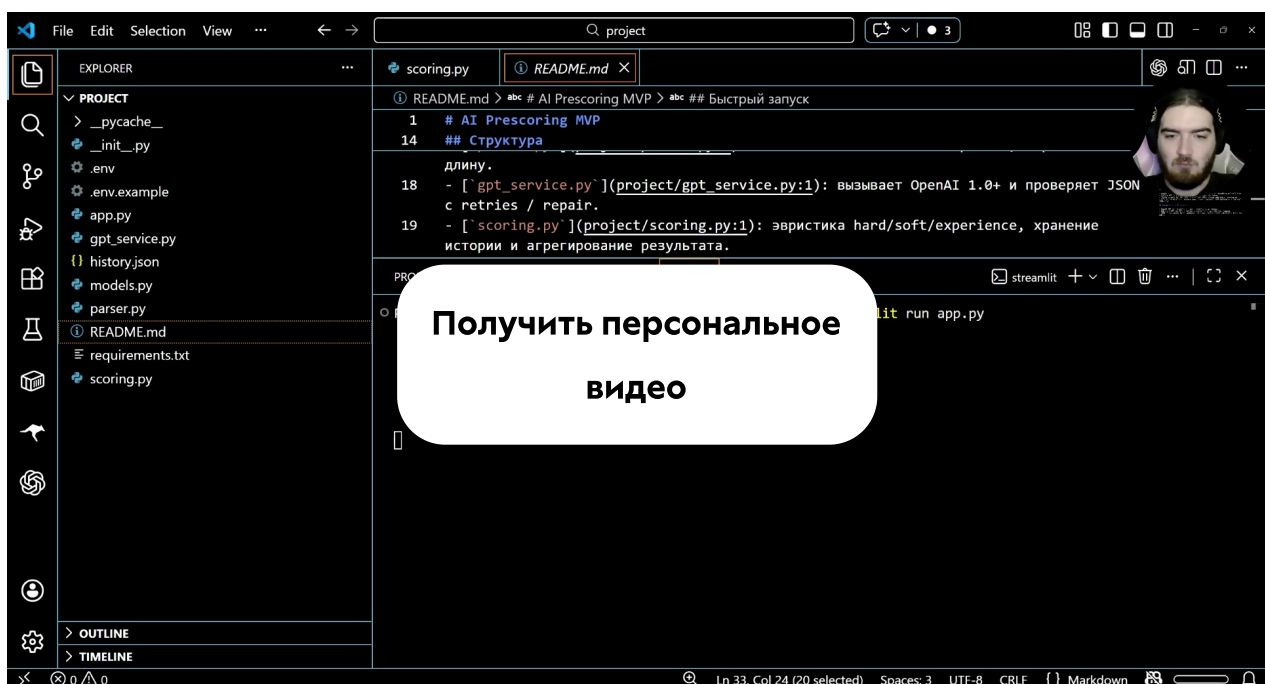
```

97 def main() -> None:
98     st.set_page_config(page_title="AI Prescoring", layout="wide")
99     st.title("ИИ-прескоринг кандидатов")
100    st.caption("Сервис для быстрой предоценки соответствия вакансии и резюме.")
101
102    with st.sidebar:
103        st.header("Настройки")
104        hard_weight = st.slider("Hard skills weight (%)", 20, 80, 60)
105        experience_weight = st.slider("Experience weight (%)", 0, 60, 25)
106        soft_weight = st.slider("Soft skills weight (%)", 0, 40, 15)
107        st.markdown("Весовые коэффициенты нормализуются автоматически.")
108
109    vacancy = st.text_area("Описание вакансии", height=220)
110    candidate_name = st.text_input("Имя кандидата", placeholder="Иван Иванов")
111    resume_upload = st.file_uploader("Загрузите PDF резюме", type=["pdf"])
112    resume_text = st.text_area("Или вставьте текст резюме", height=200)
113

```

- Не нужно ручное оформление элементов, настройка кнопок, текстовых полей и загрузки файлов проще и быстрее, чем в **Django** или **Flask**.

3. Запуск, тестирование и оптимизация ИИ-прескоринга



▼ Запуск проекта

1. Переходим в **README** и запускаем команды:

- При первом запуске вводится **e-mail** (любой).
- После запуска интерфейс загружается автоматически, без дополнительной настройки дизайна.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\daniil\Desktop\prompt\Новая папка\project> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.18.0.1:8501

```

2. Тестируем на примере вакансии и резюме:

- Загружаем описание вакансии.

Настройки

Hard skills weight (%) 60

Experience weight (%) 25

Soft skills weight (%) 15

Весовые коэффициенты нормализуются автоматически.

ИИ-прескоринг кандидатов

Сервис для быстрой предоценки соответствия вакансии и резюме.

Описание вакансии

Вакансия: Backend Developer (Python / Go)

О компании

Мы — IT-компания, разрабатывающая SaaS-решения для автоматизации бизнес-процессов, CRM-системы и AI-инструменты для HR и e-commerce. Работаем с highload-сервисами и активно внедряем AI в продукты.

Задачи

- Загружаем резюме кандидата с релевантным опытом. Кандидат соответствует требованиям практически полностью.

Иван Петров

Backend Developer (Python / Go)
Email: ivan.petrov@email.com | Telegram: @ivan_backend | GitHub: github.com/ivanpetrov

О себе

Backend-разработчик с 4+ годами опыта разработки высоконагруженных сервисов. Специализация: REST API, микросервисная архитектура, базы данных, Docker. Опыт построения CRM-систем, платежных сервисов и AI-интеграций.

Технические навыки

- Python (FastAPI, Flask, Django)
- Golang (net/http, Gin)
- PostgreSQL, Redis, MongoDB
- Docker, Docker Compose
- REST API, JWT, OAuth2
- RabbitMQ, Kafka (базовые знания)
- Git, CI/CD
- Linux, Nginx

Опыт работы

Backend Developer — IT Solutions (2021–2024)

- Разработка REST API для CRM-системы (FastAPI + PostgreSQL)
- Проектирование микросервисной архитектуры
- Интеграция платежных систем (Stripe API)
- Оптимизация SQL-запросов (снижение времени отклика на 40%)
- Настройка Docker-окружения и деплоя на VPS

- Указываем имя кандидата.

Имя кандидата

Иван Петров

Загрузите PDF резюме

Drag and drop file here

Limit 200MB per file • PDF

Backend_Developer_Candidate_Example.pdf 48.6KB

×

Или вставьте текст резюме

Browse files

- Настраиваем веса.

Вес **hard skills, experience** и **soft skills** можно менять прямо в интерфейсе. Система пересчитывает итоговую оценку без перезапуска приложения. Нажимаем **Оценить кандидата** .

 Важно соблюдать сумму весов = 100%.

Настройки

Hard skills weight (%)

60

Experience weight (%)

35

Soft skills weight (%)

5

Весовые коэффициенты нормализуются автоматически.

Drag and drop file here
Limit 200MB per file • PDF

Backend_Developer_Candidate_Example.pdf

48.6 KB

Или вставьте текст резюме

Оценить кандидата

Анализируем соответствие...

- Анализируем результат.

Результаты GPT анализа

Сильные стороны:

- 3+ года коммерческого опыта backend-разработки
- Уверенное знание Python (FastAPI / Flask / Django)
- Опыт работы с PostgreSQL
- Опыт работы с Docker
- Знание Git
- Опыт построения микросервисной архитектуры
- Понимание принципов SOLID и Clean Architecture
- Умение оптимизировать производительность

Слабые стороны:

- Отсутствие опыта с Golang
- Отсутствие опыта с Redis
- Отсутствие опыта с RabbitMQ или Kafka
- Отсутствие опыта работы с OpenAI API или другими AI-сервисами

Чего не хватает:

- Опыт с Golang
- Redis
- RabbitMQ или Kafka
- Работа с OpenAI API или другими AI-сервисами

Итоговое резюме

Кандидат обладает сильными навыками в разработке на Python, опытом работы с PostgreSQL, Docker, Git, и построением микросервисной архитектуры. Также хорошо понимает принципы SOLID и Clean Architecture. Однако ему не хватает опыта с Golang, Redis, RabbitMQ/Kafka, и работой с AI-сервисами. Рекоменд

3. Модель **gpt-3.5-turbo** показала нестабильные результаты: иногда происходит корректная оценка, а иногда заниженный скор. Для стабильности лучше заменить на более современную модель (**gpt-5-mini**).

```

105     for attempt in range(1, retries + 1):
106         try:
107             response = CLIENT.chat.completions.create(
108                 model="gpt-5-mini-2025-08-07",
109                 messages=messages,
110             )
111         except OpenAIError as error:
112             logging.exception("OpenAI API request failed")
113             if attempt == retries:
114                 raise
115             time.sleep(1)
116             continue
117

```

4. При стандартном распределении весов будет получен более высокий итоговый балл.

Настройки

Hard skills weight (%) 60

Experience weight (%) 25

Soft skills weight (%) 15

Весовые коэффициенты нормализуются автоматически.

Финальный score

63

Результаты GPT анализа

Сильные стороны: • Соответствует требованию 3+ лет коммерческого опыта (5+ лет). • Глубокие знания Python и опыт с FastAPI, Flask, Django для разработки REST API. • Опыт проектирования микросервисной архитектуры и самостоятельного проектирования сервисов. • Опыт с PostgreSQL, включая оптимизацию запросов и использование EXPLAIN ANALYZE.	Слабые стороны: • Не указаны конкретные провайдеры платёжных систем и примеры интеграций (Stripe, PayPal и т.п.). • Неявно указана глубина опыта на Go в годах — нужно уточнить масштаб и сложность Go-проектов. • Нет явного упоминания о практике мониторинга и observability (Prometheus, Grafana, Sentry и т.п.). • Нет подробностей по тестированию	Чего не хватает: • Конкретные интеграции с платёжными провайдерами (если это критично для проекта). • Инструменты оркестрации/кластера (Kubernetes) — не упомянуты явно. • Инструменты мониторинга и трассировки (Prometheus/Grafana/Jaeger) — не упомянуты явно.
---	---	---

5. Есть возможность:

- изменять веса;
- пересчитывать оценку;
- скачивать результаты;
- анализировать историю кандидатов.

История кандидатов

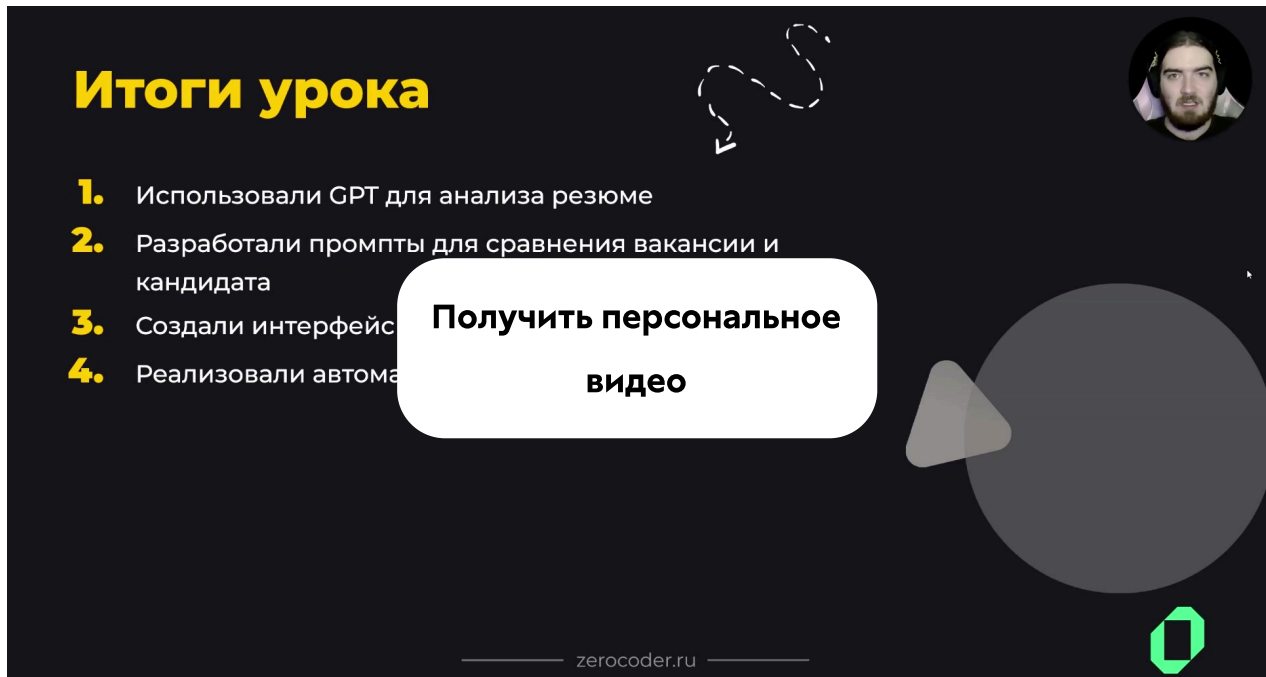
Кандидат	Финальный score	GPT score	⋮	Эвристика	Дата
Иван Иванов	64	93		64	2026-02-19 20:04
Иван Иванов	63	95		63	2026-02-19 20:02
Иван Иванов	15	9		15	2026-02-19 20:04

⚠️ Преимущества Streamlit

- Интерфейс создается без написания фронтенда.
- Не требуется ручная настройка форм и загрузки файлов.
- Быстрое создание рабочего сервиса для прескоринга кандидатов.

Проект демонстрирует, как быстро можно собрать практичный **HR-инструмент** без сложной фронтенд-разработки.

4. Результаты урока



Итоги урока

1. Использовали GPT для анализа резюме
2. Разработали промпты для сравнения вакансии и кандидата
3. Создали интерфейс
4. Реализовали автоматизацию

Получить персональное видео

zerocoder.ru

▼ Итоги урока

⚠ Автоматизация позволяет значительно ускорить работу HR:

- исключается необходимость вручную просматривать сотни и тысячи резюме;
- сокращается время на первичный отбор;
- процесс становится более системным и управляемым.

Оценка строится по трем ключевым параметрам:

- hard skills;
- soft skills;
- опыт.

Наработки можно использовать для создания собственной системы автоматизации. Возможна доработка логики и расширение функционала. Можно добавить работу с различными форматами файлов.

Можно использовать не только **OpenAI**, но и любых других провайдеров с доступом через **API**. Для подключения потребуется заменить ключ доступа и адаптировать методы запроса. Промпты можно оставить без существенных изменений. Критической разницы в выборе конкретной модели нет — использовать можно тот сервис, к которому есть доступ.

Результаты урока:

1. Использовали GPT для анализа резюме.
2. Разработали промпты для сравнения вакансии и кандидата.
3. Создали интерфейс на Streamlit.
4. Реализовали автоматический прескоринг.

Дополнительные материалы

- ▼ **Код проекта эксперта**

Код проекта эксперта на GitHub

(<https://github.com/MrGAN12009/prescore>)

Домашнее задание

Реализуй собственную систему автоматического прескоринга кандидатов на основе материалов урока.

Что нужно сделать:

1 Реализовать анализ резюме

- Настроить подключение к API (через `.env`).
- Организовать получение **строго структурированного JSON** со следующими полями: `score`, `strengths`, `weaknesses`, `missing_skills`, `summary`.
- Обеспечить обработку и “ремонт” JSON при необходимости.

2 Реализовать парсинг PDF

- Использовать `pdfplumber` для извлечения текста.

- Очистить текст (удалить лишние переносы, пробелы, шум).
- Ограничить объем текста (например, до 6000 символов).

3 Реализовать систему оценки

- Разделить оценку на:
 - hard skills
 - experience
 - soft skills
- Настроить веса (сумма = 100%).
- Реализовать расчет итогового балла по взвешенной формуле.

4 Создать интерфейс

- Реализовать веб-интерфейс на Streamlit.
- Добавить:
 - поле для ввода вакансии;
 - загрузку PDF-резюме;
 - ввод имени кандидата;
 - отображение итогового score;
 - вывод сильных и слабых сторон.

5 Реализовать хранение истории

- Добавить сохранение результатов (JSON или CSV).
- Обеспечить возможность просмотра нескольких кандидатов.

Формат сдачи

Необходимо предоставить:

1.  Ссылку на репозиторий на GitHub с проектом.
2.  Скриншоты:
 - интерфейса приложения;
 - примера анализа кандидата;
 - таблицы с историей кандидатов.
3.  Краткое описание (5–10 предложений):
 - какую модель использовали;
 - какие веса задали;
 - какие сложности возникли;
 - какие улучшения планируете добавить.

Дополнительное задание (по желанию)

- Реализовать возможность динамического изменения весов в интерфейсе.

- Сравнить результаты работы разных моделей.

Цель домашнего задания — собрать полноценный прототип HR-инструмента и закрепить навыки проектирования промптов, работы с API и построения AI-сервиса.

На следующем занятии

Перейдём к изучению нового модуля.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PErr10.7. ИИ-ассистент. Прескоринг кандидатов на вакансии (new)

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

Реализуй собственную систему автоматического прескоринга кандидатов на основе материалов урока.

ЧТО НУЖНО СДЕЛАТЬ:

1 РЕАЛИЗОВАТЬ АНАЛИЗ РЕЗЮМЕ

- Настроить подключение к API (через `.env`).
- Организовать получение **строго структурированного JSON** со следующими полями: `score`, `strengths`, `weaknesses`, `missing_skills`, `summary`.
- Обеспечить обработку и "ремонт" JSON при необходимости.

2 РЕАЛИЗОВАТЬ ПАРСИНГ PDF

- Использовать `pdfplumber` для извлечения текста.
- Очистить текст (удалить лишние переносы, пробелы, шум).
- Ограничить объем текста (например, до 6000 символов).

3 РЕАЛИЗОВАТЬ СИСТЕМУ ОЦЕНКИ

- Разделить оценку на:
 - `hard skills`
 - `experience`
 - `soft skills`
- Настроить веса (сумма = 100%).
- Реализовать расчет итогового балла по взвешенной формуле.

4 СОЗДАТЬ ИНТЕРФЕЙС

- Реализовать веб-интерфейс на Streamlit.
- Добавить:
 - поле для ввода вакансии;
 - загрузку PDF-резюме;
 - ввод имени кандидата;
 - отображение итогового `score`;
 - вывод сильных и слабых сторон.

5 РЕАЛИЗОВАТЬ ХРАНЕНИЕ ИСТОРИИ

- Добавить сохранение результатов (JSON или CSV).
- Обеспечить возможность просмотра нескольких кандидатов.

ФОРМАТ СДАЧИ

Необходимо предоставить:

1. 🔗 Ссылку на репозиторий на GitHub с проектом.
2. 🖼️ Скриншоты:
 - интерфейса приложения;
 - примера анализа кандидата;
 - таблицы с историей кандидатов.
3. 📄 Краткое описание (5–10 предложений):
 - какую модель использовали;
 - какие веса задали;
 - какие сложности возникли;
 - какие улучшения планируете добавить.

💡 ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ (ПО ЖЕЛАНИЮ)

- Реализовать возможность динамического изменения весов в интерфейсе.
- Сравнить результаты работы разных моделей.

Цель домашнего задания — собрать полноценный прототип HR-инструмента и закрепить навыки проектирования промптов, работы с API и построения AI-сервиса.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=341528207>)

Следующий урок